

⚡ IBM BOB 2.0 HACKATHON • PRINCIPAL SYSTEM PRESENTATION

Every component passed.
The system didn't.

Meridian is an autonomous release-convergence intelligence system for distributed enterprise architectures. It bridges the fatal gap between isolated CI/CD green builds and actual multi-service runtime reality.

THE CENTRAL ARCHITECTURAL DILEMMA

100% Green CI/CD ≠ Validated System

In modern enterprises, decoupled microservices, messaging queues (IBM MQ, Kafka), and legacy mainframe ledgers are deployed independently by decentralized teams.

- ✓

Unit & integration tests pass inside every team's container.
- ✓

Kubernetes liveness probes & cloud health checks report 200 OK.
- ✗

The live combination running in Production was never tested together.

CORE MISSION

RECONSTRUCT → REHEARSE → PROVE

MULTI-SERVICE CONVERGENCE RADAR

Toggle: **PROD** (Diverged)

PAYMENTS SERVICE
v2.4.0 (Matched)

ORDER INVENTORY
v4.1.2 (Matched)

MQ-BRIDGE
v3.1 (ArgoCD Synced)

LEGACY-LEDGER
v6.9 (Pending CR-4471)



FIRST DIVERGENCE DETECTED: Boundary mq-bridge 3.1 → legacy-ledger 6.9 has 0 validation records. Silent schema truncation imminent!

 THE ENTERPRISE BLINDSPOT • "THE SYSTEM NOBODY TESTED"

HTTP 200 OK. Pod Healthy.

Business Data Silently Corrupted.

How a routine Friday release with partial GitOps sync caused a catastrophic data corruption that bypassed every APM tool, alerting rule, and liveness probe in the enterprise.

THURSDAY 16:00 UTC • STAGE ENVIRONMENT

Release R-26.9 Validated End-to-End

Tested in Stage with `mq-bridge 3.1` + `legacy-ledger 7.0`. 12-character customer IDs and new currency headers functioned flawlessly. Full release sign-off achieved.

FRIDAY 11:42 UTC • PRODUCTION DISASTER

The Asynchronous Partial Promotion

Argo CD auto-sync automatically promoted `mq-bridge 3.1`. But `legacy-ledger` was held at `6.9` waiting for Sunday's 02:00 UTC maintenance window (Change Request CR-4471).

THE SILENT FAILURE TRAP:

✓ HTTP 200 ✓ MQ ACK ✓ DB COMMIT ✗ DATA CORRUPTED

FIXED-WIDTH RECORD PARSER COMPARISON

CRITICAL OFFSET SHIFT

// Raw Wire Payload emitted by mq-bridge 3.1:

CUST12345678USD0000149.50

// Ingested by legacy-ledger 6.9 (Old 10-char COBOL Copybook):

- customer_id [0:10]: "CUST123456" ← "78" SILENTLY CLIPPED!
- currency_code [10:13]: "78U" ← INVALID CURRENCY!
- amount_cents [13:25]: "SD0000149.50" → \$780,000,149.00

Result: Wrong customer billed for \$780 Million instead of \$149.50! Traditional monitoring shows zero pod crashes because the process returned status code 0.

⚙️ THE SOLUTION • DETERMINISTIC CONVERGENCE LIFECYCLE

From Post-Mortem Blame to Continuous Convergence Intelligence.

Meridian executes a closed-loop 5-stage workflow that deterministically proves software composition validity before, during, and after promotion.

01

RECONSTRUCT

Observe ground truth runtime versions

02

UNDERSTAND

Isolate unvalidated version combinations

03

REHEARSE

Simulate promotions counterfactually

04

PROVE

Execute isolated compatibility probes

05

REMEDiate

Draft backwards-compatible shims

Stage 1: RECONSTRUCT — Ground Truth Discovery

NON-INVASIVE READ-ONLY

Instead of believing git commit tags or release manifests, Meridian deploys non-mutating adapters across Kubernetes clusters, OpenShift pods, and container registries to discover what binary SHAs are actively serving traffic.

INPUT ARTIFACTS

K8s Pod Specs, Image Digests, Git SHAs

DETERMINISTIC VERDICT

Verified Environment Reality Matrix

SAFETY GUARANTEE

Zero cluster mutation, strictly read-only

 ARCHITECTURE • DETERMINISTIC CORE + IBM BOB 2.0 AGENT SWARM

Dual-Engine Architecture: Mathematical Rigor + Agentic Collaboration.

Meridian pairs a deterministic Python reconciliation engine with IBM Bob 2.0's autonomous specialist agents via 17 custom Model Context Protocol (MCP) tools.

FOUNDATION LAYER

Deterministic Python Engine

Release validation decisions must be mathematically verifiable. The core reconciliation engine runs zero LLM calls in its verdict path—preventing hallucinations and ensuring 100% reproducible results for auditors and CABs.

- ✓ **Validation Memory:** Tracks historical combinations
- ✓ **Probe Sandbox:** Subprocess isolation (no live impact)
- ✓ **PreToolUse Hooks:** Hard safety blocks on mutations

 Enforced Safety: `kubect1 apply` and `helm upgrade` blocked at hook layer.



Release Investigator Agent

Parses git commits, PR diffs, and CAB tickets (CR-4471) to detect scope.



Environment Investigator Agent

Queries live pods across DEV, TEST, STAGE, PROD to uncover actual running versions.



Implicit Contract Discovery Agent

Infers undocumented formats from legacy Excel specs, copybooks, and code.



Evidence Reviewer & Auditor Agent

Assembles cryptographically sealed Evidence Packets with SHA-256 hashes.

 SCIENTIFIC VALIDATION • PROOF, NOT PREDICTION

First Demonstrated Divergence: Executable Proof in Sandboxes.

Enterprise release managers do not accept AI predictions or probabilistic guesses. Meridian generates isolated, runnable compatibility probes that mathematically prove failures.

TERMINOLOGY DISTINCTION

Why “First Demonstrated Divergence”?

In distributed systems, the phrase "root cause" is misleading. When 8 services communicate, failures emerge at boundaries. Meridian identifies the **First Demonstrated Divergence**: the earliest edge in the business flow that lacks validation evidence AND fails an executable probe.

Evidence Packet #EV-2026-0927:

- Boundary: mq-bridge:3.1 → legacy-ledger:6.9
- Sandbox Exit Code: 1 (AssertionError)
- Artifact Hash: sha256:8f4c2b910e...
- Reproducibility: python probes/run_isolated.py

✓ 100% Deterministic: Run 1,000 times, get the exact same byte-level failure.

PROBE SANDBOX RUNNER

IDLE

▶ Test Stage (v3.1 + v7.0)

⚡ Test Prod (v3.1 + v6.9)

[meridian-engine] Probe runtime initialized in isolated subprocess.
[meridian-engine] Ready to execute wire-protocol verification...

Click a button above to run isolated probe test.

 ENTERPRISE VALUE • ZERO UNTESTED PRODUCTION DRIFT

Transforming Release Risk Into
Deterministic Release Convergence.

Meridian protects core banking, healthcare, and enterprise supply chains from silent multi-million dollar data corruption before a single customer is impacted.

0

Silent Schema Corruptions

Proactively flagged before asynchronous promotion occurs in live clusters.

< 3.2s

Counterfactual Rehearsal

Simulates entire promotion cascades without deploying a single container.

100%

Read-Only Safety

Zero write commands allowed; sandbox tests execute in isolated subprocesses.

Experience the Live Prototype in Action

Explore the complete end-to-end scenario: Reconstruct reality, run IBM Bob investigation, and review evidence.

 Open Meridian App

 Restart Deck